UWeb

Ultra Web Framework for MicroPython

Version 1.1.0 — Final Production Ready

A lightweight, secure, and feature-rich asynchronous web framework designed for MicroPython on resource-constrained devices (ESP32, RP2040, ESP8266, and similar).

English Documentation

1. Introduction

UWeb is a production-oriented asynchronous web framework written specifically for MicroPython. It offers a clean Flask-like API while carefully managing memory, providing built-in security features, authentication, WebSocket support, static/media serving, and input validation — all optimized for microcontrollers with limited RAM.

1.1 Key Features

- Async HTTP server based on **uasyncio**
- Routing with typed path parameters (`<id:int>`, `<name:str>`, `<path:path>`)
- Built-in static & media serving (`/static/`, `/media/`, `/assets/`)
- WebSocket support with rooms, broadcast, ping/pong and heartbeat
- JWT-style token authentication + refresh tokens + session management
- Input validation (SQL injection & XSS pattern detection, type checking)
- Per-IP rate limiting and IP blacklisting
- CORS support and automatic security headers
- Simple template engine (`{{variable}}`)
- Multipart form & file upload handling
- Middleware system
- Memory optimized (`__slots__`, controlled GC, limited caches)

2. Requirements

- MicroPython 1.19+ (1.22 or newer recommended)
- Standard modules: `uasyncio`, `ujson`, `uhashlib`, `ubinascii`
- Recommended free RAM: 100–150 KB for comfortable operation

Tested platforms:

- ESP32 / ESP32-S3
- Raspberry Pi Pico W (RP2040)
- ESP8266 (limited — reduce `MAX_REQUEST_SIZE` and rate limits)

3. Installation

1. Copy `uweb.py` to your device (root or a `lib/` folder).
2. Directories are created automatically, but you may pre-create them:

```
import os
os.mkdir("templates")
os.mkdir("static")
os.mkdir("media")
```

4. Quick Start

```
from uweb import UWeb
import uasyncio as asyncio

app = UWeb(secret_key="your-super-secret-key-change-me", debug=True)
```

```
@app.get("/")
def home(req):
    return app.html("<h1>Hello from UWeb!</h1>")

@app.get("/api/hello/<name:str>")
def hello(req, name):
    return {"message": f"Hello, {name}!"}

@app.post("/api/echo")
def echo(req):
    data = req.json()
    return {"received": data}

asyncio.run(app.run(host="0.0.0.0", port=80))
```

5. Routing

Routes are registered with decorators. Path parameters support types **str** (default), **int**, **float**, and **path** (catch-all remaining segments).

```
@app.route("/users/<id:int>", methods=["GET", "PUT", "DELETE"])
def user_handler(req, id):
    ...

@app.get("/files/<path:path>")      # multi-segment path
def files(req, path):
    ...

@app.post("/login")
@app.put("/items/<item_id:int>")
@app.delete("/items/<item_id:int>")
@app.patch("/profile")
@app.ws("/ws")                      # WebSocket endpoint
```

6. Request Object

The request object provides convenient access to method, path, headers, body, client IP, query parameters, JSON/form data, and uploaded files.

```
req.method
req.path
req.query_params
req.headers
req.body
req.client_ip
req.user_agent
req.user                # populated after authentication

req.json()              # parse JSON body
req.json(schema={...})  # with lightweight schema validation
req.form()              # form-urlencoded or multipart
req.query("key", default=None)
req.header("Authorization")
req.file("avatar")      # returns dict with filename, data, size
```

7. Response Helpers

```
app.json({"ok": True}, status=200)
app.html("<h1>Page</h1>")
```

```
app.text("plain text")
app.file(binary_data, filename="report.pdf")
app.redirect("/login")
app.error(404, "Not found")
app.render_template("index.html", {"title": "Home"})
```

You may also return a plain `dict`/`list` (automatically converted to JSON) or a full `Response` object.

8. Authentication

UWeb includes a complete token-based authentication system with access tokens, refresh tokens, in-memory sessions, role checks, and token revocation.

```
# Login helper
@app.post("/login")
def login(req):
    data = req.json()
    # verify credentials...
    return app.login(user_id=123, extra_data={"roles": ["admin"]})

# Protect a route
@app.get("/profile")
def profile(req):
    result = app.require_auth(req)
    if isinstance(result, Response):
        return result          # 401 response
    return {"user_id": req.user["user_id"]}

# Role-based access
@app.get("/admin")
def admin(req):
    result = app.require_role(req, ["admin"])
    if isinstance(result, Response):
        return result
    return {"secret": "data"}
```

Clients send the token as: `Authorization: Bearer <access_token>`

9. WebSockets

```
@app.ws("/chat")
async def chat(ws, req):
    await ws.send({"type": "welcome", "msg": "Connected"})
    while ws.connected:
        msg = await ws.receive()
        if msg is None:
            break
        if isinstance(msg, dict) and msg.get("type") == "pong":
            continue
        await app.ws_manager.broadcast("general", msg)
```

WebSocket methods: `send()`, `receive()`, `ping()`, `close()`.
Manager helpers: `broadcast(room, message)`, `get_client_count()`, `get_room_clients(room)`.

10. Static & Media Files

The following routes are registered automatically:

URL Prefix	Directory	Description
/static/...	static/	CSS, JS, images, fonts
/assets/...	static/	Alias of static
/media/...	media/	User uploads / generated files

```
@app.post("/upload")
def upload(req):
    f = req.file("file")
    if f:
        ok = app.upload_media(f["filename"], f["data"])
        return {"ok": ok}
    return app.error(400, "No file")
```

11. Middleware & CORS

11.1 Middleware

```
def auth_middleware(req, app):
    if req.path.startswith("/api/private"):
        result = app.require_auth(req)
        if isinstance(result, Response):
            return result
    return None   # continue to next middleware / handler

app.use(auth_middleware)
```

11.2 CORS

```
app.enable_cors(
    origins=["https://myfrontend.com", "http://localhost:3000"],
    methods=["GET", "POST", "PUT", "DELETE"],
    headers=["Content-Type", "Authorization"]
)
```

12. Configuration Options

```
app = UWeb(
    secret_key="change-me-in-production",
    rate_limit=60,                # requests per minute per IP
    templates_dir="templates",
    static_dir="static",
    media_dir="media",
    debug=False
)

app.max_request_size = 256 * 1024   # optional override
app.blacklisted_ips.add("1.2.3.4")  # block an IP
```

13. Security Features

- Input sanitization against common SQL injection and XSS patterns
- Automatic security headers on every response (X-Frame-Options, nosniff, XSS-Protection, Referrer-Policy, Cache-Control)
- Per-IP rate limiting

- IP blacklisting
- HttpOnly + SameSite=Strict cookies
- Path-traversal protection on static/media and file operations
- In-memory token blacklisting (jti-based)

14. Limitations (MicroPython Reality)

- Template engine is basic (only `{{var}}` replacement — no loops or conditionals)
- Sessions and token blacklist live only in RAM (lost on reboot)
- Multipart parser is lightweight — keep uploads reasonably small
- Maximum concurrent WebSockets limited by available RAM (default 50)
- No built-in HTTPS (use a reverse proxy or MicroPython SSL where available)

15. License

Designed and optimized for real-world MicroPython deployments.
Happy coding on the edge!